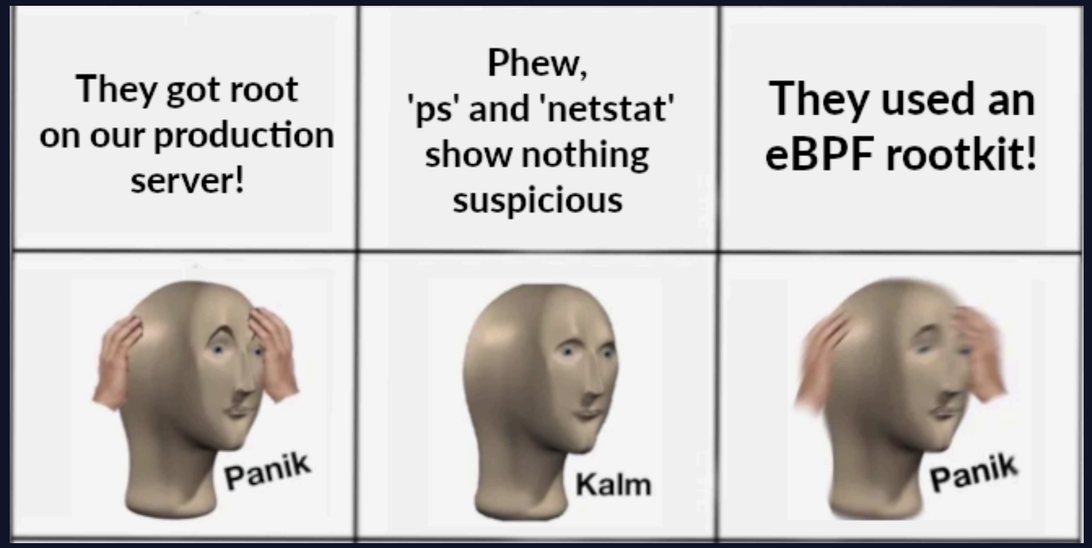
kernel 4/4


They got root on our production server!
Phew, 'ps' and 'netstat' show nothing suspicious
They used an eBPF rootkit!
Panik
Kalm
Panik

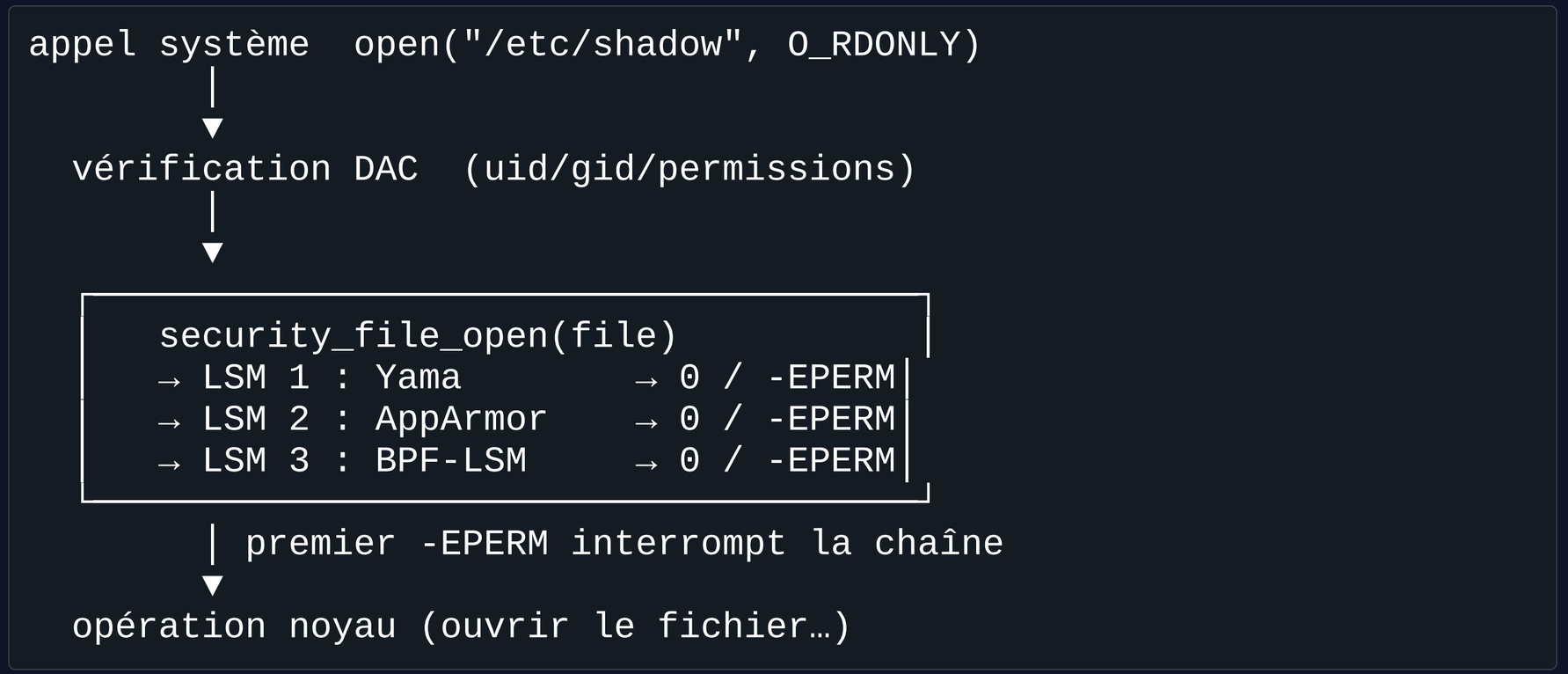
Au programme

- Linux Security Modules (LSM)
- eBPF : la machine virtuelle du noyau
- BPF-LSM / KRSI
- Hooks utiles pour un EDR
- Emprisonnement de processus

Linux Security Modules

- Framework de sécurité **mandataire** (MAC) intégré au noyau — Linux 2.6
- Hooks insérés directement dans le chemin d'exécution du noyau
- Appelés **après** la vérification DAC (permissions UNIX)
- Retour `0` = autoriser, valeur négative = refuser (`-EPERM` , `-EACCES` ...)
- Ne modifie pas la sémantique des appels système — ajoute une couche de contrôle

Architecture LSM



LSMs existants et stacking



- **SELinux** (NSA) — politique étiquettes/contextes, très granulaire
- **AppArmor** (Canonical) — politique par chemin de fichier
- **Yama** — restrictions ptrace et dumpable
- **Smack** — étiquettes simples, embarqué/IoT
- **BPF-LSM** (Google, Linux 5.7) — programmes eBPF comme LSM ← *notre sujet*

Stacking depuis Linux 4.x — plusieurs LSM actifs simultanément :

```
# dans /boot/cmdline ou CONFIG_LSM=
lsm=yama,apparmor,bpf
```

L'ordre définit l'ordre d'appel dans la chaîne de décision.

Familles de hooks LSM

Famille	Exemples de hooks
Processus	`bprm_check`, `task_alloc`, `task_fix_setuid`
Fichiers	`file_open`, `path_mkdir`, `path_unlink`, `inode_create`
Réseau	`socket_create`, `socket_connect`, `socket_bind`
Modules	`kernel_module_request`, `kernel_read_file`
VFS	`sb_mount`, `sb_remount`
IPC	`msg_queue_msgsnd`, `sem_semop`

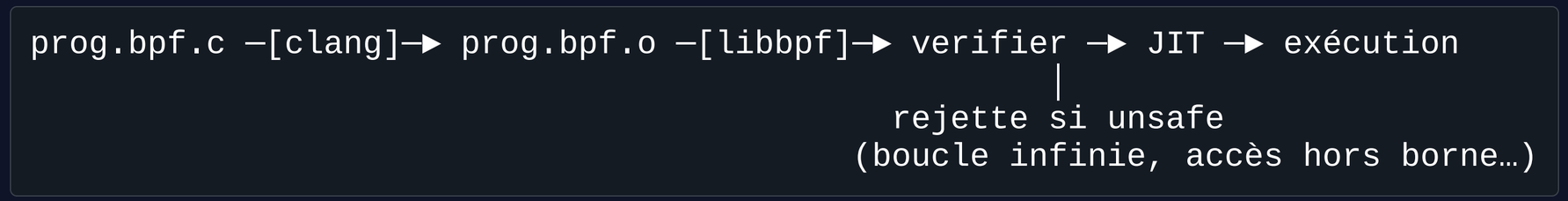
Plus de **240 hooks** dans Linux 6.x

→ `include/linux/lsm_hook_defs.h`

→ commentaire "fonctionnel" dans security/security.c, cherchez pour X la fonction security_X

eBPF : la machine virtuelle du noyau

- **Extended Berkeley Packet Filter** — Linux 3.18+, généralisé en 4.x
- Bytecode vérifié **statiquement** avant chargement (le *verifier*)
- **JIT compilé** vers le code natif x86/ARM/RISC-V au chargement
- Sandboxé : pas de boucles infinies, accès mémoire contrôlés
- Hot-reload : pas de recompilation noyau, pas de redémarrage



eBPF : la machine virtuelle du noyau

Le format `bytecode` eBPF se comporte comme un langage machine pour une "machine eBPF".
La "machine eBPF" est une machine virtuelle à registre:

- 11 registres 64 bits (de R0 à R10).
- une stack de taille fixe égale à 512 octets.

Facile lors de la compilation "Juste à temps" (JIT) de faire correspondre les registres de cette VM avec les registres "réels" de l'architecture.

eBPF : Le `Verifier`



Analyse du CFG (Control Flow Graph) du programme:

- Détection de boucle infinie, seul les boucles bornés sont autorisés.
- Limite d'instructions (1 millions max)
- Détection de code mort, le code non atteignable est rejeté.

Exécution symbolique du code avant chargement:

- Suivi de l'état et type de registre
- Contrôle de la pile (empêche le dépassement des 512)
- Fuites d'informations (empêche de lire s'il n'y a pas eu écriture avant)

eBPF : Les outils



`BPF` nécessite que le compilateur dispose du bon "backend". Actuellement seul Clang/LLVM posséde cette "fausse" architecture.

Sur mint

```
$ sudo apt update
$ sudo apt install -y clang llvm libbpf-dev linux-tools-common linux-headers-$(uname -r)
```

On va avoir besoin de générer un .h contenant toutes les structures/types de notre kernel en cours.

```
$ bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
$ vim trace_bpf.c # on code, voir slide suivant
```

eBPF : Un programme d’exemple



```c
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>

// On définit la structure du contexte pour sys_enter_execve
// En respectant les offsets vus dans le fichier "format" du noyau
struct execve_args {
    unsigned long long unused; // offset 0: common_type, flags, preempt_count, pid
    int syscall_nr;            // offset 8: __syscall_nr
    int padding;               // Padding naturel 32 bits pour l'alignement 64 bits
    const char *filename;      // offset 16: le pointeur vers la chaîne en espace utilisateur
};

SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct execve_args *ctx) {
    // On alloue un petit buffer sur la pile (stack) eBPF (rappel: max 512 octets !)
    char nom_programme[128];

    // On copie la chaîne depuis l'espace utilisateur vers notre buffer eBPF
    // Arguments : destination, taille max, source
    long res = bpf_probe_read_user_str(nom_programme, sizeof(nom_programme), ctx->filename);

    // Si la lecture a réussi (res renvoie la longueur de la chaîne copiée)
    if (res > 0) {
        bpf_printk("Nouveau lancement : %s\n", nom_programme);
    } else {
        bpf_printk("Erreur de lecture du nom du programme\n");
    }

    return 0;
}
```

Compilation d'un programme de test: boucle_bpf.c

```
$ clang -O2 -g -target bpf -c trace_bpf.c -o trace_bpf.o
```

eBPF : Un programme d’exemple



On peut charger notre programme au point de rendez-vous `/sys/fs/bpf/ma_trace`

```
$ sudo bpftool prog load trace_bpf.o /sys/fs/bpf/ma_trace autoattach
```

La fonction d'aide "bpf_printk" envoie les informations sur `/sys/kernel/debug/tracing/trace` mais aussi dans le même répertoire dans le pipe `trace_pipe`.

```
$ sudo cat /sys/kernel/debug/tracing/trace_pipe
```

Puis dans un autre terminal, lancez des commandes `/bin/ls`, `/usr/bin/cat`, ...

eBPF : Un programme d'exemple



Attention: vérifiez que les traces sont bien active si le `cat .../trace_pipe` ne retourne rien:

```
$ sudo cat /sys/kernel/debug/tracing/tracing_on
1
$ cat /sys/kernel/debug/tracing/options/trace_printk
1
```

Vous pouvez consulter les programmes `BPF` chargés, ou les programmes attachés et donc actifs:

```
$ sudo bpftool prog show
...
$ sudo bpftool link show
...
```

eBPF : Un programme d’exemple



Vous pouvez voir à quoi ressemble votre programme `eBPF` compilé:

```
$ llvm-objdump -d trace_bpf.o

sample/tracebin_bpc.o:  file format elf64-bpf

Disassembly of section tracepoint/syscalls/sys_enter_execve:

0000000000000000 <trace_execve>:
       0:       79 13 10 00 00 00 00 00 r3 = *(u64 *)(r1 + 0x10)
       1:       bf a1 00 00 00 00 00 00 r1 = r10
       2:       07 01 00 00 80 ff ff ff r1 += -0x80
       3:       b7 02 00 00 80 00 00 00 r2 = 0x80
       4:       85 00 00 00 72 00 00 00 call 0x72
       5:       b7 01 00 00 01 00 00 00 r1 = 0x1
       6:       6d 01 07 00 00 00 00 00 if r1 s> r0 goto +0x7 <LBB0_2>
...
```

eBPF : Plus d’infos



man bpf(2): Point d'entrée de la documentation pour eBPF version `low-level`.

bpf-helpers(7): Liste toutes les fonctions `utiles` pour les programmes `eBPF`.

Exemples:

```
u64 bpf_get_current_pid_tgid(void)

              Description
                     Get the current pid and tgid.

              Return A 64-bit integer containing the current tgid  and  pid,  and  created  as  such:  current_task->tgid  <<  32  |  cur-
                     rent_task->pid.
```

Types de programmes BPF

`SEC(...)`	Point d'attache
`tracepoint/...`	points d'instrumentation statiques noyau
`kprobe/...`	toute fonction noyau (dynamique)

Ces types de programmes eBPF sont surtout là pour effectuer des traces.

Types de programmes BPF



Les `tracepoint` sont des points de contrôle statique dans le kernel et exposés par `/sys/kernel/debug/tracing/events/` .

```
$ sudo tree /sys/kernel/debug/tracing/events/
```

Chaque sous dossier est une catégorie inspectable, ex: `SEC("tracepoint/sched/sched_process_exit")`

Les `kprobe` correspondent aux symboles publiques du noyau tel que vous auriez pu le hooker via l'API Kprobe en C. ex: `SEC("kprobe/vfs_read")` ou `SEC("kretprobe/tcp_v4_connect")`

Types de programmes BPF

`SEC(...)`	Point d'attache
`xdp`	réseau (avant IP stack)
`tc`	réseau (après IP stack)
`perf_event`	compteurs hardware/software

Ici on hook tout le sous-système, ex: `SEC("xdp")`. C'est au chargement que cela change. Pour XDP ou TC on connect à l'interface physique:

```
sudo bpftool net attach xdp id 123 dev eth0
```

Types de programmes BPF

`SEC(...)`	Point d'attache
`lsm/hook`	hook Linux Security Module
`lsm.s/hook`	hook LSM **sleepable** (`bpf_d_path` , etc.)

Le suffixe `.s` marque un programme sleepable, c'est à dire qu'on peut appeler des helpers qui nécessitent un contexte de sommeil (e.g. `bpf_d_path`) sinon `kernel panic` .

Structure d’un programme BPF



```c
// prog.bpf.c
#include "vmlinux.h" // tous les types noyau (CO-RE): Code Once-Run Everywhere
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

char LICENSE[] SEC("license") = "GPL";

SEC("lsm/file_open")
int BPF_PROG(file_open_hook, struct file *file)
{
    char comm[16];
    bpf_get_current_comm(comm, sizeof(comm));
    bpf_printk("open par : %s\n", comm);
    return 0;   // 0 = autoriser
}
```

`BPF_PROG` : macro qui déclare le programme avec la signature correcte selon le type de hook (évite de manipuler `struct pt_regs *` manuellement).

BPF Maps — introduction

Structures de données **persistantes dans le noyau**, partagées entre :

- les invocations successives d'un programme BPF
- le programme BPF et l'espace utilisateur

Type	Usage typique
`BPF_MAP_TYPE_HASH`	table clé→valeur (politique par PID)
`BPF_MAP_TYPE_ARRAY`	tableau indexé (compteurs, config)
`BPF_MAP_TYPE_RINGBUF`	file d'événements kernel→userspace
`BPF_MAP_TYPE_LRU_HASH`	hash avec éviction automatique
`BPF_MAP_TYPE_PERCPU_ARRAY`	compteurs sans contention par CPU

BPF Maps — introduction

L'usage avancé de `eBPF` nécessitant des `BPF Maps` requiert un changement d'approche:

- On compile toujours le code vers l'architecture `bpf`
- On traduit les bytecodes en une version C pure via `bpftool` dans un `.h` à inclure.
- On code un programme normale sur x86_64 qui charge le programme tout en étant actif et communique avec le BPF actif via les `BPF Maps`

BPF Maps — déclaration côté BPF (.bpf.c)

```c
// Exemple d'une map pour stocker un PID et une valeur dans le cas d'un LSM
struct {
    __uint(type,        BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key,         u32);   // type de la clé   : un PID
    __type(value,       u64);   // type de la valeur : des flags
} policy_map SEC(".maps");
```

- `SEC(".maps")` : place la map dans la section ELF `.maps` reconnue par libbpf au chargement
- `__uint`, `__type` : macros BTF qui encodent les types pour CO-RE
- La map est **créée par libbpf** lors de l'appel à `prog__open_and_load()` dans le programme en userland, pas par le code BPF lui-même

BPF Maps — opérations côté BPF

```
u32 pid = bpf_get_current_pid_tgid() >> 32;

// Lookup : retourne un pointeur vers la valeur, NULL si clé absente
u64 *val = bpf_map_lookup_elem(&policy_map, &pid);
if (!val) return 0;   // toujours tester avant de déréférencer !
// utiliser *val ici
bpf_printk("la valeur %d\n", *val);
...
// Update (possible depuis le BPF aussi)
u64 new_val = 1;
bpf_map_update_elem(&policy_map, &pid, &new_val, BPF_NOEXIST);
...
// Delete
bpf_map_delete_elem(&policy_map, &pid);
```

BPF Maps — opérations côté BPF

Flag	Comportement
`BPF_ANY`	créer **ou** remplacer
`BPF_NOEXIST`	créer seulement si la clé est **absente**
`BPF_EXIST`	remplacer seulement si la clé **existe**

BPF Maps — accès depuis l'espace utilisateur

Deux façons d'obtenir un file descriptor vers la map :

```c
#include <bpf/libbpf.h>
#include "prog.skel.h" // générer par bpftool - voir dans qq slides

// ── fournis par prog.skel.h ──
struct prog *skel = prog__open_and_load();
int map_fd = bpf_map__fd(skel->maps.policy_map);

// ── Épingler la map pour la partager avec un autre processus ──
bpf_map__pin(skel->maps.policy_map, "/sys/fs/bpf/policy_map");

// ── Depuis un autre processus (outil de config, jailer…) ──
int map_fd = bpf_obj_get("/sys/fs/bpf/policy_map");
if (map_fd < 0) { perror("bpf_obj_get"); return 1; }
```

BPF Maps — accès depuis l’espace utilisateur

L'épinglage (`pin`) persiste tant que le fichier existe dans `/sys/fs/bpf/` .

`bpf_obj_get` permet à un processus séparé de manipuler la map sans recharger le BPF.

BPF Maps — opérations depuis l'espace utilisateur



```c
#include <bpf/bpf.h>   // bpf_map_update_elem, bpf_map_lookup_elem…

// Toujours dans l'exemple d'un LSM travaillant sur des couples PID/valeur
u32 key = 1234;   // PID cible
u64 value = 0x3;    // flags de politique

// Insérer ou mettre à jour
int r = bpf_map_update_elem(map_fd, &key, &value, BPF_ANY);
if (r < 0) perror("update");

// Lire (copie la valeur dans result)
u64 result = 0;
bpf_map_lookup_elem(map_fd, &key, &result);

// Supprimer (libérer un process de la prison)
bpf_map_delete_elem(map_fd, &key);
```

Les fonctions retournent `0` en succès, `-1` en erreur (`errno` positionné).

Ring Buffer — kernel → userspace

Transport d'événements depuis le BPF vers l'espace utilisateur.

```c
// ── déclaration ──
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 1 << 24);   // 16 MiB
} events SEC(".maps");

// ── envoi depuis le BPF ──
struct event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
if (!e) return 0;
e->pid = bpf_get_current_pid_tgid() >> 32;
bpf_get_current_comm(e->comm, sizeof(e->comm));
bpf_ringbuf_submit(e, 0);
```

Ring Buffer — kernel → userspace

Transport d'événements depuis le BPF vers l'espace utilisateur.

```
// ── lecture côté userspace ──
struct ring_buffer *rb = ring_buffer__new(
    bpf_map__fd(skel->maps.events), handle_event, NULL, NULL);
while (1)
    ring_buffer__poll(rb, 100 /* ms */);
```

CO-RE : Compile Once – Run Everywhere



Problème : les offsets des structures noyau varient entre versions du noyau.

Solution CO-RE :

1. Le compilateur enregistre les *relocations BTF* dans le `.bpf.o`
2. libbpf applique les relocations au chargement selon le BTF du noyau cible
3. Nécessite `CONFIG_DEBUG_INFO_BTF=y`

```
# Générer vmlinux.h depuis le noyau en cours d'exécution
$ bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
```

CO-RE : accès aux structures noyau côté BPF



`bpf_get_current_task_btf()` est un **helper BPF** : il ne peut être appelé que depuis un programme BPF en cours d'exécution dans le noyau.

```c
// prog.bpf.c  ← compilé vers le bytecode BPF
#include "vmlinux.h"          // contient la définition de struct task_struct
#include <bpf/bpf_core_read.h>

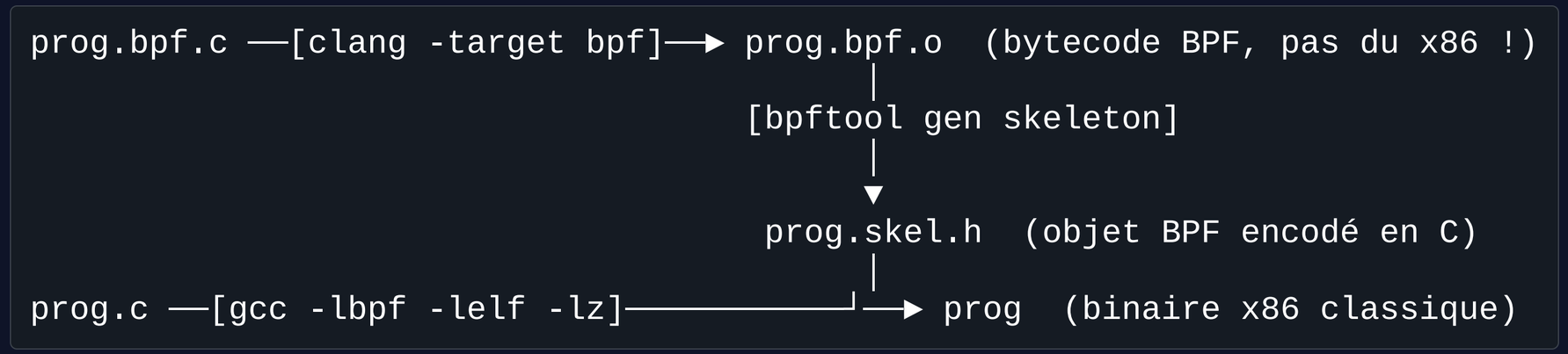
SEC("lsm/file_open")
int BPF_PROG(my_hook, struct file *file)
{
    // helper BPF : retourne un pointeur noyau vers la task_struct courante
    struct task_struct *task = (struct task_struct *)
        bpf_get_current_task_btf();

    // BPF_CORE_READ : lecture avec relocation BTF
    // → résiste aux changements de layout entre versions du noyau
    // BPF_CORE_READ(ptr, champ1, champ2) est l'équivalent BPF-safe de ptr->champ1->champ2
    u32 ppid = BPF_CORE_READ(task, real_parent, tgid);

    bpf_printk("ppid=%d\n", ppid);
    return 0;
}
```

BPF Skeleton — deux binaires distincts

La compilation BPF produit **deux binaires de nature différente** :



- `prog.bpf.c` → cross-compilé vers le **bytecode BPF** (`-target bpf`)
- `prog.skel.h` → généré par bpftool : contient l'objet BPF **intégré en dur** dans le .h et les fonctions `prog__open_and_load()` , `prog__attach()` , `prog__destroy()`
- `prog.c` → compilé normalement en x86, lie contre **libbpf** (`-lbpf`)

BPF Skeleton — Makefile type

```
CLANG   ?= clang
CC      ?= gcc
BPFTOOL ?= bpftool
ARCH    := $(shell uname -m | sed 's/x86_64/x86/')

# 1. Cross-compiler le code BPF vers le bytecode BPF (pas du x86 !)
prog.bpf.o: prog.bpf.c vmlinux.h
        $(CLANG) -g -O2 -target bpf -D__TARGET_ARCH_$(ARCH) \
                 -I/usr/include -c $< -o $@

# 2. Générer le skeleton C (embarque l'objet BPF dans un .h)
prog.skel.h: prog.bpf.o
        $(BPFTOOL) gen skeleton $< > $@

# 3. Compiler le programme userspace normalement (x86), lier libbpf
prog: prog.c prog.skel.h
        $(CC) -g -O2 -Wall -o $@ $< -lbpf -lelf -lz
```

Dépendances système : `clang llvm libbpf-dev bpftool`

BPF Skeleton — programme userspace

```c
// prog.c  (compilé en x86, exécuté par root)
#include <stdio.h>
#include <signal.h>
#include "prog.skel.h"    // généré par bpftool : contient tout le BPF

static volatile int running = 1;
static void on_signal(int s) { running = 0; }

int main(void) {
    struct prog *skel;

    // Charge le bytecode BPF dans le noyau + vérifie via le verifier
    skel = prog__open_and_load();
    if (!skel) { perror("load"); return 1; }

    // Attache les programmes aux hooks déclarés dans le .bpf.c
    prog__attach(skel);

    // Accès aux maps : skel->maps.policy_map, skel->maps.events…
    int map_fd = bpf_map__fd(skel->maps.policy_map);

    signal(SIGINT, on_signal);
    while (running)
        pause();  // ou ring_buffer__poll(rb, 100)

    prog__destroy(skel);   // détache les hooks + libère la mémoire
    return 0;
}
```

BPF-LSM — KRSI

Kernel Runtime Security Instrumentation — Linux 5.7 (mai 2020)

Attache des programmes BPF aux hooks LSM. La valeur de retour du programme BPF **est** la décision de sécurité.

```
CONFIG_BPF=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_LSM=y
CONFIG_DEBUG_INFO_BTF=y
CONFIG_LSM="yama,bpf"     # ou via cmdline : lsm=yama,bpf
```

BPF-LSM — KRSI

Propriétés clés :

- Hot-reload sans redémarrage
- Pas de module signé requis
- Le verifier garantit l'absence de crashs noyau
- Accès complet aux structures noyau via CO-RE/BTF

BPF-LSM vs alternatives

	BPF-LSM	SELinux policy	LKM LSM
Hot-reload	✅	❌	❌
Module signé requis	❌	❌	✅ (Secure Boot)
Accès contexte noyau	✅ complet	limité	✅ complet
Sécurité du mécanisme	✅ verifier	✅ politique	⚠️ aucune
Programmabilité	✅ C + maps	langage dédié	✅ C noyau
Cas d'usage	EDR, monitoring	MAC statique	recherche

Hooks EDR : surveillance des processus

```c
// Exécution d'un binaire (execve)
SEC("lsm/bprm_check_security")
int BPF_PROG(check_exec, struct linux_binprm *bprm)
{
    char comm[16];
    bpf_get_current_comm(comm, sizeof(comm));
    bpf_printk("exec: %s\n", comm);
    return 0;
}

// Élévation de privilèges (setuid/setgid)
SEC("lsm/task_fix_setuid")
int BPF_PROG(check_setuid, struct cred *new,
             const struct cred *old, int flags)
{
    u32 old_uid = BPF_CORE_READ(old, uid.val);
    u32 new_uid = BPF_CORE_READ(new, uid.val);
    if (old_uid != 0 && new_uid == 0)
        bpf_printk("PrivEsc pid=%d!\n",
                   (u32)(bpf_get_current_pid_tgid() >> 32));
    return 0;
}
```

Hooks EDR : accès aux fichiers



`bpf_d_path` requiert un programme **sleepable** (`lsm.s/`)

```c
SEC("lsm.s/file_open")
int BPF_PROG(check_file_open, struct file *file)
{
    char path[256];
    // bpf_d_path : résout le chemin complet depuis un struct path*
    bpf_d_path(&file->f_path, path, sizeof(path));
    bpf_printk("open: %s\n", path);
    return 0;
}

// Suppression de fichier (détection ransomware)
SEC("lsm/path_unlink")
int BPF_PROG(check_unlink, const struct path *dir,
             struct dentry *dentry)
{
    char name[64];
    bpf_probe_read_kernel_str(name, sizeof(name),
        BPF_CORE_READ(dentry, d_name.name));
    bpf_printk("unlink: %s\n", name);
    return 0;
}
```

Hooks EDR : réseau & modules

```c
// Connexion réseau sortante
SEC("lsm/socket_connect")
int BPF_PROG(check_connect, struct socket *sock,
             struct sockaddr *address, int addrlen)
{
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    // inspecter address->sa_family, sa_data (port + IP)...
    bpf_printk("connect pid=%d\n", pid);
    return 0;   // ou -EPERM pour bloquer
}

// Chargement de module noyau (anti-rootkit)
SEC("lsm/kernel_module_request")
int BPF_PROG(check_kmod, char *kmod_name)
{
    bpf_printk("insmod: %s\n", kmod_name);
    // return -EPERM;  // bloquer tout chargement de module
    return 0;
}
```

Informations sur le processus courant

```
// PID / TID
u64 id  = bpf_get_current_pid_tgid();
u32 pid = id >> 32;      // PID (process)
u32 tid = (u32)id;       // TID (thread)

// Nom du processus (comm, max 16 octets)
char comm[TASK_COMM_LEN];
bpf_get_current_comm(comm, sizeof(comm));

// UID / GID
u64 ugid = bpf_get_current_uid_gid();
u32 uid  = (u32)ugid;

// Accès à la task_struct (CO-RE)
struct task_struct *task = (struct task_struct *)
    bpf_get_current_task_btf();
u32 ppid = BPF_CORE_READ(task, real_parent, tgid);
```

Emprisonnement de processus

Architecture : BPF map comme base de politique dynamique

```
userspace (jailer)                BPF program (kernel)
─────────────────────             ──────────────────────────────────────

jail <pid> /etc   →               jail_map[pid] = { flags }

                                  SEC("lsm.s/file_open")
                                  → pid = bpf_get_current_pid_tgid() >> 32
                                  → policy = jail_map[pid]
                                  → si policy && path interdit
                                  →     return -EPERM
```

Granularité possible : PID, cgroup ID, UID, comm

Emprisonnement : le hook

```c
#define JAIL_BLOCK_ETC   (1U << 0)
#define JAIL_BLOCK_PROC  (1U << 1)

struct jail_policy { u32 flags; };

SEC("lsm.s/file_open")
int BPF_PROG(jail_file_open, struct file *file)
{
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    struct jail_policy *p =
        bpf_map_lookup_elem(&jail_map, &pid);
    if (!p) return 0;

    char path[256];
    bpf_d_path(&file->f_path, path, sizeof(path));

    if ((p->flags & JAIL_BLOCK_ETC) &&
            bpf_strncmp(path, 5, "/etc/") == 0)
        return -EPERM;

    if ((p->flags & JAIL_BLOCK_PROC) &&
            bpf_strncmp(path, 6, "/proc/") == 0)
        return -EPERM;

    return 0;
}
```

`bpf_strncmp` disponible depuis Linux 5.17 — sinon, boucle bornée.

Comparaison Landlock / seccomp / BPF-LSM

	BPF-LSM	Landlock	seccomp
Qui pose la politique	root (CAP_BPF)	tout processus	tout processus
Granularité	chemin, socket, inode…	répertoire/droits	numéro syscall
Accès au contexte	✅ complet	❌	args syscall
Décision dynamique	✅ via maps	❌ statique	❌ statique
Monitoring	✅ ring buffer	❌	❌
Cas d'usage EDR	✅ monitoring + blocage	sandbox app	sandbox app

EDRs utilisant BPF-LSM

Des solutions industrielles reposent sur ces mêmes mécanismes :

- **Falco** (CNCF) — détection d'intrusion via eBPF/LSM
- **Tetragon** (Cilium/Isovalent) — observabilité + enforcement BPF-LSM
- **KubeArmor** (CNCF) — policy enforcement pour conteneurs/Kubernetes
- **ebpf-security** (Google) — recherche interne à l'origine de KRSI

Structure commune : programme BPF attaché aux hooks → ring buffer → daemon userspace (analyse, alerte, blocage).